

Hearth Notes

Init Date: 06/29/2026

Author: Sun

06/29/2026

X86 Exercism - ([link](#))

Using 'neg' to get the negative opposite of some bytes

```
mov rax, 4          ; moves 0100 into the rax register
neg rax             ; transforms to twos-complement -> 1100 (-4)
```

Using 'adc' for addition that's too big for the register

X86 Exercism - ([link](#))

```
mov al, 0xFF; loads 255 (0xFF) into 8-bits (AL)
mov bl, 0x01; load 1 (0x01) into 8-bits (BL)

clc          ; clears the carry flag! (garbage memory removed)

add al, bl   ; attempt to add them, it will overflow!
              ; the carry flag now holds 0x01 because 256 is too big

xor al, al   ; zero-out AL

adc al, 0x01; you'll get 2, because the carry flag was holding a 1
              ; adc = 1 + 1
```

Using 'inc' and 'dec' to add or remove '1' to a register

```
mov al, 0x01; move 0x01 (1) into an 8-bit register
inc al      ; 1 -> 2
dec al      ; 2 -> 1
```

Hearth Notes

Init Date: 06/29/2026

Author: Sun

Multiplication uses both halves of a register

using 'imul' aka multiplication, for 64 bit numbers, uses the top half of rax and the bottom half of rdx, this means you must save whatever is in rdx first before doing multiplication, because it will get erased and used for the operation

Moving values to larger registers using 'movzx'

```
mov ax, 1024; eax is 32-bits; ax is the lower 16-bits
              ; 1024 = 0000 0100 0000 0000

mov cx, 2      ; ecx is 32-bits; cx is the lower 16-bits
              ; 2 = 0000 0000 0000 0010

movzx eax, ax   ; 1024 stays in the lower 16-bits, but clears the upper 16
              ; ax → eax → 0000...(3-words)...0000 0100 0000 0000

movzx ecx, cx   ; same thing, just with '2'

mul ecx         ; top-half and bottom-half of the register are used for...
              ; multiplication. This correctly holds '1024 * 2 = 2048'
              ; mul = unsigned multiplication
              ; imul = signed multiplication (twos-compliment)
```

Next thing